



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
Disciplina: ALGORITMO E ESTRUTURA DE DADOS I

Semestre: 2026.2

Data: ____/____/____
PROFESSORA: Rosana Rego

— Exercício 2: Unidade I —

Enumerações (**enum**) em vetores dinâmicos de structs

Nome: _____

Nota: _____ /1,0

Objetivo

Este documento descreve o funcionamento do programa `atividade2.c`, evolução da `atividade1.c`, cujo objetivo é demonstrar, de forma visual e interativa (usando a biblioteca **raylib**), os seguintes conceitos de linguagem C:

- Definição de um tipo **enum** (`TipoMoeda`) para representar categorias;
- Uso do **enum** como campo dentro de uma *struct* (`Moeda`);
- **Alocação dinâmica** de um vetor de *structs* com `malloc` e liberação com `free`;
- **Ponteiro para struct** (`Moeda *`) passado para funções que alteram o dado original (marcar a moeda como coletada);
- Uso de `switch` sobre um valor de **enum** para decidir cor e valor de cada moeda.

O programa exibe um jogador (círculo azul) que se move pela tela e coleta moedas de três raridades diferentes, cada uma valendo uma pontuação distinta.

1 Estrutura de dados: a moeda

Cada moeda é representada por uma *struct* que combina posição, raio, um campo do tipo **enum** e o estado de coleta:

```
1 typedef enum {  
2     MOEDA_BRONZE ,  
3     MOEDA_PRATA ,  
4     MOEDA_OURO  
5 } TipoMoeda;  
6  
7 typedef struct {  
8     Vector2    pos;  
9     float      raio;  
10    TipoMoeda  tipo;  
11    int        valor;  
12    bool       coletada;  
13 } Moeda;
```

O **enum** `TipoMoeda` não guarda nenhum dado extra: ele apenas nomeia três constantes inteiras (`MOEDA_BRONZE = 0`, `MOEDA_PRATA = 1`, `MOEDA_OURO = 2`), tornando o código muito mais legível do que se usássemos números mágicos diretamente.

2 Decidindo cor e valor a partir do enum

Duas funções recebem um `TipoMoeda` e usam `switch` para decidir, respectivamente, a cor de desenho e a pontuação da moeda:

```
1 Color corDaMoeda(TipoMoeda tipo) {
2     switch (tipo) {
3         case MOEDA_BRONZE: return (Color){160, 90, 40, 255};
4         case MOEDA_PRATA:  return (Color){190, 190, 190, 255};
5         case MOEDA_OURO:   return GOLD;
6         default:           return WHITE;
7     }
8 }
9
10 int valorDaMoeda(TipoMoeda tipo) {
11     switch (tipo) {
12         case MOEDA_BRONZE: return 5;
13         case MOEDA_PRATA:  return 10;
14         case MOEDA_OURO:   return 25;
15         default:           return 0;
16     }
17 }
```

Esse é o principal ganho do **enum**: cada nova categoria de moeda só exige um novo `case`, sem espalhar constantes numéricas pelo programa.

3 O vetor dinâmico de moedas (`Moeda *`)

Assim como o vetor de bolas da `atividade1.c`, o vetor de moedas é um único bloco contíguo de memória, alocado com uma só chamada de `malloc`:

```
1 Moeda *criarMoedas(int quantidade) {
2     Moeda *moedas = (Moeda *)malloc(quantidade * sizeof(Moeda));
3     for (int i = 0; i < quantidade; i++) {
4         Moeda *m = (moedas + i);
5         m->pos = (Vector2){ GetRandomValue(30, LARGURA_JANELA -
6                               30),
7                               GetRandomValue(30, ALTURA_JANELA -
8                               30) };
9         m->raio = 10.0f;
10        m->tipo = (TipoMoeda)GetRandomValue(MOEDA_BRONZE,
11                                              MOEDA_OURO);
12        m->valor = valorDaMoeda(m->tipo);
13        m->coletada = false;
14    }
15    return moedas;
16 }
```

4 Coleta via ponteiro para struct

A função `tentarColetar` recebe um **ponteiro para struct** (`Moeda *`) e altera diretamente o campo `coletada` do elemento do vetor original – sem essa passagem por ponteiro, a alteração se perderia ao final da chamada:

```
1 bool tentarColetar(Moeda *m, Vector2 posJogador, float raioJogador) {
2     if (m->coletada) return false;
```

```

3
4     float dx = m->pos.x - posJogador.x;
5     float dy = m->pos.y - posJogador.y;
6     float distancia = (dx * dx + dy * dy);
7     float somaRaios = (m->raio + raioJogador) * (m->raio +
8         raioJogador);
9
10    if (distancia <= somaRaios) {
11        m->coletada = true;
12        return true;
13    }
14    return false;
}

```

Ciclo de vida da memória

Etapa	Vetor de moedas (Moeda *)
Alocação	criarMoedas()
Uso	tentarColetar() + desenharMoeda()
Liberação	free(moedas)

Toda memória alocada com `malloc` deve, obrigatoriamente, ser liberada com `free` antes do fim do programa – caso contrário, ocorre um *memory leak* (vazamento de memória).

5 Exercícios

Exercício 1 — Moedas que reaparecem

Atualmente, uma vez coletada, a moeda desaparece definitivamente (`coletada = true` para sempre).

Tarefa: modifique o programa para que, 3 segundos depois de coletada, a moeda reapareça em uma nova posição aleatória.

Dicas:

- Adicione um campo `float tempoColeta` à *struct* `Moeda` para guardar o instante da coleta.
- Use `GetTime()` do `raylib` para obter o tempo atual em segundos.
- Em `tentarColetar`, ao marcar `coletada = true`, grave também `m->tempoColeta = GetTime()`.
- A cada quadro, percorra o vetor e, se `m->coletada` for verdadeiro e `GetTime() - m->tempoColeta >= 3.0f`, sorteie nova posição e marque `m->coletada = false`.

Exercício 2 — Nova raridade: moeda de diamante

Atualmente o **enum** `TipoMoeda` possui apenas três valores.

Tarefa: adicione um novo valor `MOEDA_DIAMANTE`, valendo 50 pontos e desenhada com uma cor diferente das demais.

Dicas:

- Acrescente `MOEDA_DIAMANTE` ao final do **enum** `TipoMoeda`.
- Adicione um novo **case** em `corDaMoeda` e em `valorDaMoeda`.
- Em `criarMoedas`, ajuste o intervalo de `GetRandomValue` para incluir o novo valor.
- Torne a moeda de diamante mais rara reduzindo a chance de ela ser sorteada (por exemplo, sorteando um número extra entre 0 e 9 e só permitindo diamante se esse número for 0).